



Lecture 7: Derived State

INTERVIEW QUESTION

[> CORE](#)

How do you compute a value that updates automatically when its inputs change?

You have a cart total that should recalculate whenever a price or quantity changes. Writing a function to recompute it by hand feels wrong.

State is the source of truth for your application. But often, the values you display in the UI aren't the raw state itself, but rather a *computation* based on that state.

The Problem with Manual Recalculation

Imagine you are building a checkout page with a list of items, and you need to display the grand total. If you create a standalone `$state(0)` variable for the total, you must manually update it every single time the cart changes.

When a user adds an item, you must call `updateTotal()`. When they remove an item, you must call `updateTotal()`. When they apply a 10 percent discount code, you must call `updateTotal()`. If another developer joins the team and adds a feature to change an item's quantity but forgets to call `updateTotal()`, the cart goes out of sync. Your user gets charged the wrong amount, resulting in a severe customer service nightmare.

The underlying problem is that the cart data and the total are fundamentally disconnected. The total *depends* on the cart, but your imperative code treats them as independent variables.

The `$derived` Rune

To fix this, Svelte 5 introduces the `$derived` rune. It allows you to declare a variable that is automatically calculated from other reactive variables.

● ● ● Checkout.svelte

```
1  <script>
2    let price = $state(20);
3    let quantity = $state(2);
4
5    let total = $derived(price * quantity);
6  </script>
7
8  <button onclick={() => quantity++}>
9    Add Item (Total: ${total})
10 </button>
```

Automatically stays in SYNC

When you declare `let total = $derived(price * quantity)`, you are telling the Svelte compiler to treat this value as entirely dependent on `price` and `quantity`.

You never have to manually update `total`. Whenever `price` or `quantity` changes, Svelte knows it needs to recalculate `total` before updating the DOM.

KEY TAKEAWAY

If `$state` is the source of truth, `$derived` is the automatic reflection of that truth.

Complex Derivations with `$derived.by`

Sometimes your computation is too complex to fit on a single line, like iterating over an array to sum up prices. For these cases, you can use `$derived.by`, which accepts a function block.



```
1 <script>
2   let cart = $state([10, 20, 30]);
3
4   let grandTotal = $derived.by(() => {
5     let sum = 0;
6     for (const item of cart) {
7       sum += item;
8     }
9     return sum;
10  });
11 </script>
```

*Iterates and sums up the **REACTIVE** array*

Push-Pull Reactivity

Svelte uses a highly optimized system called *push-pull reactivity*. When a `$state` variable changes, Svelte immediately pushes a notification to everything that depends on it, marking those derived values as dirty. However, it doesn't actually do the mathematical recalculation yet. It waits until the exact moment the UI tries to pull or read the value. This ensures Svelte never wastes CPU cycles calculating values that are never displayed on screen.

Furthermore, if the newly calculated value happens to be identical to the old value, Svelte halts the update right there, preventing unnecessary DOM updates.

INTERVIEW TIP

To impress the interviewer: Mention that `$derived` replaces Svelte 4's reactive statements (the `$:` label). While the old syntax felt like magic, the new `$derived` rune behaves like a standard JavaScript variable declaration, making it much easier to reason about scope and refactor into separate `.svelte.js` files.

Summary

When a value in your UI depends on other reactive data, never update it manually. Instead, use the `$derived` rune (or `$derived.by` for complex logic). Svelte's compiler automatically tracks the dependencies and ensures the derived value is recalculated exactly when needed. This guarantees your UI is never out of sync with your underlying state, completely eliminating a massive class of bugs.

	MANUAL (Vanilla JS)	SVELTE 5 (<code>\$derived</code>)
Declaration	<code>let total = 0;</code>	<code>let total = \$derived(p * q);</code>
Updating	Must call <code>update()</code> manually	Automatically tracked by compiler
Performance	Runs whenever you explicitly call it	Optimized push-pull recalculation